NumPy & Numerical Computing

Session 11 study guide · Python 3 · VS Code · 14 days × 3 fulfilment hubs

In Session 09 we stored records in lists and dictionaries and looped over them one at a time. That is the right tool for business records. But when the data is a **grid of numbers**, such as units dispatched per day per hub, looping one value at a time becomes slow to write and easy to get wrong. **NumPy** stores numbers with an explicit **shape**, so one expression can calculate across the whole collection at once.

This guide follows the Session 11 deck. It starts with a short dictionary and set refresher, then makes the move from lists to NumPy arrays explicit with side-by-side comparisons, before working through one dataset: 14 days of dispatch across three hubs. Every example was run on Python 3 with NumPy and Matplotlib; the output shown is the real output.

List or NumPy array? The comparison at a glance

Question	Python list	NumPy array
What can it hold?	Anything, mixed: `["ORD-1001", 4, 250.0]`	One data type for every element (usually numbers)
What does `+` do?	**Joins** two lists: `[1, 2] + [3, 4]` → `[1, 2, 3, 4]`	**Adds** matching positions: → `[4 6]`
What does `* 2` do?	**Repeats** the list: `[1, 2, 1, 2]`	**Doubles** every value: `[2 4]`
Maths on every value	a loop or comprehension	one expression, e.g. `prices * 1.15` (**vectorised**)
Shape	no shape; a grid is a list of lists	explicit `.shape`, e.g. `(14, 3)`, with numbered axes
A cell in a grid	`grid[1][2]`	`grid[1, 2]`
A whole column	a comprehension: `[row[2] for row in grid]`	`grid[:, 2]`
Filtering	comprehension with `if`	a Boolean mask: `values[values > 120]`
Growing one item at a time	`.append()` is quick and natural	fixed size; `np.append` builds a new array
Totals and statistics	`sum()`, `max()`, a loop	`.sum()`, `.mean()`, `.std()`, with `axis=` for rows or columns
Best for	records, mixed data, collections that grow or change	numerical grids, one calculation across many values, charts, ML

> **Rule of thumb:** Use a **list** (or dictionary) for *records* and changing collections. Use a **NumPy array** when you have *numbers in a regular shape* and want to calculate across them.

Part	Topic	What you will do
0	Refresher: dictionaries and sets	update, remove, look up, set operations
1	Bridge: from lists to arrays	same `+`, different meaning; two order totals in one line
2	List or array? Side by side	nine comparisons and when to choose each
3	Array foundations	shape, ndim, size, dtype; creating arrays
4	Indexing, slicing and filtering	a day, a hub, a week; Boolean masks; views and copies
5	Vectorisation and broadcasting	compare 14 days with 3 hub benchmarks
6	Reshape, split, stack	42 readings into a grid; two weeks
7	Aggregation	totals by axis, busiest day, counts, spread
8	Transfer: a photograph	the same ideas on pixels
9	Charts with Matplotlib	line, bar, scatter
10	Practical	the 14-day dispatch review

Lesson plan (3 hours)

Time	Block	You do (trainer)	Learners do	Slides
0–10	Part 0 · Refresher	Quick dictionary and set operations on screen (optional if time is short).	Try it yourself 0.	older Day 4 deck
10–25	Parts 1–2 · Lists to arrays	Predict `+` for lists and arrays. Walk the side-by-side comparisons.	Try it yourself 1–2.	1–7
25–43	Part 3 · Foundations	Shape, ndim, size, dtype; constructors.	Predict shapes; Try it yourself 3.	8–13
43–61	Part 4 · Indexing	Day 2 is row 1. Masks keep the grid shape.	Try it yourself 4.	14–22
61–81	Part 5 · Broadcasting	The shape rule from the right; the wrong-order trap.	Try it yourself 5.	23–31
81–91	Break			32
91–106	Part 6 · Reshape	42 values into (14, 3); split and stack check.	Try it yourself 6.	33–36
106–122	Part 7 · Aggregation	The axis you name disappears. argmax is a position.	Try it yourself 7.	37–43
122–132	Part 8 · Photograph	Demo only: shape, crop, safe brightening.	Watch and predict.	44–49
132–143	Part 9 · Charts	Line, bar, scatter; label every chart.	Try it yourself 9.	50–55
143–175	Part 10 · Practical	Coach; check shapes and meaning.	Complete the starter script.	56–62
175–180	Close	Findings, one limitation, next session.	Explain-it questions.	63–67

Before class

☐ In the VS Code terminal, install the two libraries once: `pip install numpy matplotlib` (Windows may need `py -m pip install numpy matplotlib`).

☐ Check they work: `python -c "import numpy, matplotlib; print(numpy.__version__)"`.

☐ A folder `s11_numpy` opened in VS Code, with `s11_dispatch_starter.py` and `beach.jpg` from the class bundle.

☐ Charts: in a `.py` script, `plt.show()` opens a window and the script pauses until you close it. `plt.savefig("name.png")` saves the chart instead.

Teaching moves for this session

Move	How it sounds in class
Predict the shape	"Before you run it: what shape will this be?"
Name the axes	"Axis 0 is days, axis 1 is hubs. The array does not know that; we do."
Right shape is not right meaning	"It ran without an error. Is the hub order still right?"
Position versus label	"argmax said 11. Which day is that?"
List or array?	"Is this a record, or a grid of numbers?"

Part 0 Refresher: dictionaries and sets

Before moving to NumPy, a quick recap of the two structures from Session 09 that NumPy does **not** replace. Dictionaries hold named fields; sets hold unique values.

Dictionaries: add, update, remove, look up

> **Business case:** A retailer keeps monthly sales per branch, keyed by branch name.

p0_dict.py

```
branch_sales = {"Sandton": 125000, "Rosebank": 98000, "Soweto": 76000}

branch_sales["Midrand"] = 88000                          # add one key
branch_sales.update({"Rosebank": 101500, "Durban": 93000})  # add or replace several
removed = branch_sales.pop("Soweto")                     # remove and return the value
del branch_sales["Durban"]                               # remove permanently

print(branch_sales)
print("Removed Soweto:", removed)
print("Is Sandton a branch?", "Sandton" in branch_sales)
print(list(branch_sales.keys()))
print(list(branch_sales.values()))
```

OUTPUT

```
{'Sandton': 125000, 'Rosebank': 101500, 'Midrand': 88000}
Removed Soweto: 76000
Is Sandton a branch? True
['Sandton', 'Rosebank', 'Midrand']
[125000, 101500, 88000]
```

Operation	What it does
`d[key] = value`	adds a new key, or replaces the value of an existing one
`d.update(other)`	adds or replaces several keys from another dictionary
`d.pop(key)`	removes the key and **returns** its value
`d.popitem()`	removes and returns the **last** key-value pair as a tuple
`del d[key]`	removes the key permanently
`key in d`	checks whether a **key** exists
`d.keys()`, `d.values()`, `d.items()`	views of the keys, the values, or both

p0_dict_more.py

```
branch_sales = {"Sandton": 125000, "Rosebank": 101500, "Midrand": 88000}

sorted_by_name = dict(sorted(branch_sales.items()))
target_gap = {branch: 110000 - sales for branch, sales in branch_sales.items()}

print(sorted_by_name)
print(target_gap)
```

OUTPUT

```
{'Midrand': 88000, 'Rosebank': 101500, 'Sandton': 125000}
{'Sandton': -15000, 'Rosebank': 8500, 'Midrand': 22000}
```

A **dictionary comprehension** (`{key: value for ...}`) builds a dictionary in one line, just like the list comprehensions from Session 09.

Sets: unique members and set operations

> **Business case:** Marketing wants to know which customers bought the home-loan product, the credit card, or both.

p0_sets.py

```
home_loan = {"Thandi", "Pieter", "Ayesha", "Sipho"}
credit_card = {"Ayesha", "Sipho", "Lerato"}

print("Both products:", sorted(home_loan & credit_card))
print("Either product:", sorted(home_loan | credit_card))
print("Home loan only:", sorted(home_loan - credit_card))

credit_card.add("Pieter")
credit_card.discard("Nobody")   # no error if missing
credit_card.remove("Lerato")    # error if missing
print("Card holders now:", sorted(credit_card))
```

OUTPUT

```
Both products: ['Ayesha', 'Sipho']
Either product: ['Ayesha', 'Lerato', 'Pieter', 'Sipho', 'Thandi']
Home loan only: ['Pieter', 'Thandi']
Card holders now: ['Ayesha', 'Pieter', 'Sipho']
```

Operator	Name	Gives
`a & b`	intersection	values in **both**
`a | b`	union	values in **either**
`a - b`	difference	values in `a` but **not** in `b`

`.remove()` raises a `KeyError` if the value is missing; `.discard()` quietly does nothing. Dictionary keys support the same operators: `d1.keys() & d2.keys()` gives the keys two dictionaries share.

TRY IT YOURSELF

(a) Start with `stock = {"bread": 40, "milk": 25, "eggs": 60}`. Add `"butter": 15`, update milk to 30, remove eggs with `.pop()` and print what was removed. Then build `reorder = {item: qty < 30 for item, qty in stock.items()}`.

(b) `branch_a = {"C01", "C02", "C03", "C05"}` and `branch_b = {"C02", "C04", "C05"}` are customer IDs at two branches. Print the customers shared by both, all customers, and those only at branch A.

Part 1 Bridge: from lists to arrays

In Session 10 a function returned the final amount for **one** order. The retailer soon needs the same calculation for hundreds of orders or many days. The rule is unchanged:

p1_rule.py

```
print(4 * 250 * 0.90 * 1.05)   # order 1: quantity 4
print(5 * 250 * 0.90 * 1.05)   # order 2: quantity 5
```

OUTPUT

```
945.0
1181.25
```

Discount first (×0.90), then tax (×1.05). Tiny floating-point differences in the last digit are normal; round for display.

The same + means two different things

PREDICT FIRST

What does `+` do for two lists? And for two NumPy arrays?

p1_plus.py

```
import numpy as np

print([1, 2] + [3, 4])
print(np.array([1, 2]) + np.array([3, 4]))
```

OUTPUT

```
[1, 2, 3, 4]
[4 6]
```

For lists, `+` **concatenates**: it joins two sequences into a longer one. For arrays, `+` works **element-wise**: position 0 is 1 + 3, position 1 is 2 + 4. `import numpy as np` is the standard way to load NumPy; `np` is the conventional short name.

Both order totals from one expression

p1_orders.py

```
import numpy as np

quantities = np.array([4, 5])
unit_prices = np.array([250.0, 250.0])
discount_rate = 0.10
tax_rate = 0.05

order_totals = quantities * unit_prices * (1 - discount_rate) * (1 + tax_rate)
print(order_totals)
```

OUTPUT

```
[ 945.   1181.25]
```

Position	quantities	unit_prices	order_totals
0 (order 1)	4	250.0	945.00
1 (order 2)	5	250.0	1181.25

Watch out: Alignment is the meaning. NumPy knows positions, not business identities. If order 1's quantity sat beside order 2's price, NumPy would still give a number, just a wrong one.

Part 2 List or array? Side-by-side comparisons

Each comparison below does the same business task twice: once with a list, once with a NumPy array. Watch for where the list needs a loop and the array does not, and where the list is actually the better tool.

1. Add VAT to every price

Business case: A shop has three prices and must add 15% VAT to each.

p2_vat_list_error.py

```
prices = [100.0, 250.0, 80.0]
print(prices * 1.15)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s11_numpy\p2_vat_list_error.py", line 2, in <module>
    print(prices * 1.15)
          ~~~~~~~^~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

p2_vat_list.py

```
prices = [100.0, 250.0, 80.0]
with_vat = [price * 1.15 for price in prices]
print(with_vat)
```

OUTPUT

```
[114.99999999999999, 287.5, 92.0]
```

p2_vat_array.py

```
import numpy as np

prices = np.array([100.0, 250.0, 80.0])
print(prices * 1.15)
```

OUTPUT

```
[115.  287.5  92. ]
```

A list cannot be multiplied by a decimal; you must loop. The array applies `* 1.15` to every value in one expression. This is called **vectorisation**.

2. `* 2` repeats a list but doubles an array

p2_times2.py

```
import numpy as np

units = [3, 5]
print(units * 2)
print(np.array(units) * 2)
```

OUTPUT

```
[3, 5, 3, 5]
[ 6 10]
```

> **Watch out:** `units * 2` gives no error with a list. It silently repeats the list, which is a wrong answer if you meant "double the units".

3. Profit = sales − costs, month by month

p2_profit_list.py

```
sales = [42000, 38500, 51000]
costs = [30000, 29000, 33500]
profit = [s - c for s, c in zip(sales, costs)]
print(profit)
```

OUTPUT

```
[12000, 9500, 17500]
```

p2_profit_array.py

```
import numpy as np

sales = np.array([42000, 38500, 51000])
costs = np.array([30000, 29000, 33500])
print(sales - costs)
print("Total profit:", (sales - costs).sum())
```

OUTPUT

```
[12000  9500 17500]
Total profit: 39000
```

With lists you pair values with `zip()` and loop. Arrays subtract matching positions directly, and `.sum()` is built in.

4. Filter: which sales were above 40 000?

p2_filter_list.py

```
sales = [42000, 38500, 51000, 39900, 47250]
print([s for s in sales if s > 40000])
```

OUTPUT

```
[42000, 51000, 47250]
```

p2_filter_array.py

```
import numpy as np

sales = np.array([42000, 38500, 51000, 39900, 47250])
print(sales > 40000)
print(sales[sales > 40000])
```

OUTPUT

```
[ True False  True False  True]
[42000 51000 47250]
```

The array version first builds a **mask** of True/False values, then uses it to pick the values. Part 4 uses masks on a whole grid.

5. Mixed data: where lists win

> **Business case:** One order record has an ID (text), a quantity (whole number) and a price (decimal).

p2_mixed.py

```
import numpy as np

order_as_list = ["ORD-1001", 4, 250.0]
print(order_as_list, [type(v).__name__ for v in order_as_list])

order_as_array = np.array(["ORD-1001", 4, 250.0])
print(order_as_array, order_as_array.dtype)
```

OUTPUT

```
['ORD-1001', 4, 250.0] ['str', 'int', 'float']
['ORD-1001' '4' '250.0'] <U32
```

The list keeps each value's own type. The array forces **one** type on everything, so the numbers became text (`<U32` means a string of up to 32 characters) and can no longer be used in calculations. **Records with mixed fields belong in dictionaries or lists, not arrays.** Session 12's pandas handles labelled, mixed tables.

6. A grid of numbers: a cell, a row, a column

p2_grid_list.py

```
grid = [[102, 89, 117],
        [110, 94, 121],
        [115, 93, 118]]

print(grid[1][2])                    # Day 2, West
print(grid[1])                       # Day 2, all hubs
print([row[2] for row in grid])      # West, every day: needs a loop
```

OUTPUT

```
121
[110, 94, 121]
[117, 121, 118]
```

p2_grid_array.py

```
import numpy as np

grid = np.array([[102, 89, 117],
                 [110, 94, 121],
                 [115, 93, 118]])

print(grid[1, 2])      # Day 2, West
print(grid[1, :])      # Day 2, all hubs
print(grid[:, 2])      # West, every day
print(grid.shape)
```

OUTPUT

```
121
[110  94 121]
[117 121 118]
(3, 3)
```

A list of lists has rows, but getting a **column** needs a loop. An array has two axes, so a column is as easy as a row.

7. Totals by row and by column

p2_totals_list.py

```
grid = [[102, 89, 117], [110, 94, 121], [115, 93, 118]]
day_totals = [sum(row) for row in grid]
hub_totals = [sum(row[i] for row in grid) for i in range(3)]
print(day_totals, hub_totals)
```

OUTPUT

```
[308, 325, 326] [327, 276, 356]
```

p2_totals_array.py

```
import numpy as np

grid = np.array([[102, 89, 117], [110, 94, 121], [115, 93, 118]])
print(grid.sum(axis=1), grid.sum(axis=0))
```

OUTPUT

```
[308 325 326] [327 276 356]
```

8. Growing a collection: where lists win again

> **Business case:** A cashier records each sale as it happens; nobody knows how many there will be.

p2_grow.py

```
import numpy as np

sales_list = []
sales_list.append(120.5)
sales_list.append(89.99)
print(sales_list)

sales_array = np.array([120.5])
bigger = np.append(sales_array, 89.99)
print(sales_array, bigger)
```

OUTPUT

```
[120.5, 89.99]
[120.5] [120.5   89.99]
```

`list.append()` changes the list in place and is the natural way to collect items one by one. `np.append` does **not** change the original; it builds a whole new array every time. **Collect in a list, then convert once** with `np.array(sales_list)` when you are ready to calculate.

9. Speed: a balanced note

p2_speed.py

```
import time
import numpy as np

values = list(range(1_000_000))
start = time.perf_counter()
total_list = sum(v * v for v in values)
list_seconds = time.perf_counter() - start

array = np.arange(1_000_000)
start = time.perf_counter()
total_array = int((array * array).sum())
array_seconds = time.perf_counter() - start

print("Same answer:", total_list == total_array)
print(f"List:  {list_seconds:.4f} s")
print(f"Array: {array_seconds:.4f} s")
```

OUTPUT

```
Same answer: True
List:  0.0467 s
Array: 0.0037 s
```

On a million numbers the array is much faster on our machine (your timings will differ). With a handful of values the difference is meaningless, and speed alone is never the reason to choose. Choose by the **shape and meaning** of the data.

When to use which

Situation	Choose	Why
Order records with ID, customer, quantity, price	list of dictionaries	fields have names and different types
Collecting transactions as they arrive	list	`.append()` grows it cheaply
Distinct regions, membership checks	set	uniqueness is the point
Units per day per hub (14 × 3 numbers)	NumPy array	regular grid; totals by row or column
Apply VAT, a discount or a ratio to every value	NumPy array	one vectorised expression
Statistics, charts, machine learning inputs	NumPy array	the libraries expect arrays
A table with named columns and mixed types	pandas (Session 12)	labels and mixed types together

TRY IT YOURSELF

Monthly sales are `[42000, 38500, 51000, 47250]` and costs are `[30000, 29000, 33500, 31000]`.

(a) Using **lists**, compute the profit for each month and the months with profit above 15 000. (b) Do the same with **arrays**. (c) Explain in one sentence why `[42000, 38500] * 1.15` fails but works after `np.array(...)`.

Part 3 Array foundations

An array has **values**, a **shape** and, from us, a **meaning**.

p3_axes.py

```
import numpy as np

one_axis = np.array([8, 9, 10])
two_axes = np.array([[8, 9, 10],
                     [11, 12, 13]])
print(one_axis, one_axis.shape)
print(two_axes)
print(two_axes.shape)
```

OUTPUT

```
[ 8  9 10] (3,)
[[ 8  9 10]
 [11 12 13]]
(2, 3)
```

One axis: positions run in one direction, shape `(3,)`. Two axes: rows and columns, shape `(2, 3)`. In NumPy, *dimension* means the number of axes, not physical size.

Four questions to ask of any array

p3_attrs.py

```
import numpy as np

tiny = np.array([[8, 9, 10],
                 [11, 12, 13]])
print(tiny.shape)
print(tiny.ndim)
print(tiny.size)
print(tiny.dtype)
```

OUTPUT

```
(2, 3)
2
6
int64
```

Attribute	Tells you	For `tiny`
`shape`	the length of each axis, as a tuple	`(2, 3)`: 2 rows, 3 columns
`ndim`	the number of axes	2
`size`	the total number of values	6 = 2 × 3
`dtype`	the type of every element	an integer type (`int64` here; may be `int32` on some systems)

Predict before you run

p3_predict.py

```
print(tiny[0, 0])
print(tiny[1, 2])
print(tiny[0, :], tiny[0, :].shape)
print(tiny[:, 1], tiny[:, 1].shape)
```

OUTPUT

```
8
13
[ 8  9 10] (3,)
[ 9 12] (2,)
```

`(3,)` is a one-dimensional array of three values; the comma marks a one-item tuple. It differs from `(1, 3)`, which has two axes: one row and three columns.

Creating arrays

p3_create.py

```
import numpy as np

print(np.array([4, 5, 6]))
print(np.arange(1, 5))
print(np.zeros((2, 3)))
print(np.ones(3))
print(np.linspace(0, 1, 3))
```

OUTPUT

```
[4 5 6]
[1 2 3 4]
[[0. 0. 0.]
 [0. 0. 0.]]
[1. 1. 1.]
[0.  0.5 1. ]
```

Code	Creates	The argument means
`np.array([4, 5, 6])`	the values you supply	the data
`np.arange(1, 5)`	1 up to, **not including**, 5 (like `range`)	from where to where
`np.zeros((2, 3))`	a 2 × 3 grid of zeros	a **shape**
`np.ones(3)`	three ones	a shape (one axis)
`np.linspace(0, 1, 3)`	3 evenly spaced values, **both** ends included	how many samples

p3_days.py

```
import numpy as np

days = np.arange(1, 15)
print(days)
```

OUTPUT

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14]
```

One common dtype

p3_dtype.py

```
import numpy as np

measurements = np.array([1, 2.5])
print(measurements)
print(measurements.dtype)
```

OUTPUT

```
[1.  2.5]
float64
```

The list `[1, 2.5]` holds an int and a float. The array converts both to one floating type so every value is represented the same way. That makes arithmetic predictable, and it is why mixed records (comparison 5 in Part 2) do not belong in arrays.

TRY IT YOURSELF

Predict `shape`, `ndim` and `size`, then check: (a) `np.zeros((3, 4))` (b) `np.arange(1, 32)` (the days of a month) (c) `np.linspace(0, 100, 5)` (d) `np.array([[1, 2], [3, 4], [5, 6]])`.

Part 4 The dispatch data: indexing, slicing and filtering

Business case: A retailer records the units dispatched each day by three fulfilment hubs over 14 days. Each hub has a nominal planned level per day: East 110, Central 95, West 120. (The data are synthetic.)

Day	East	Central	West	Day	East	Central	West
1	102	89	117	8	105	90	116
2	110	94	121	9	112	97	119
3	115	93	118	10	108	95	120
4	107	100	119	11	116	99	126
5	119	98	125	12	121	103	131
6	92	84	109	13	96	85	112
7	87	80	104	14	91	83	106

p4_setup.py

```
import numpy as np

hub_names = ["East Hub", "Central Hub", "West Hub"]
days = np.arange(1, 15)
daily_capacity = np.array([110, 95, 120])
dispatch = np.array(raw_dispatch_units).reshape(14, 3)

print(dispatch.shape)
print(dispatch[:3])
```

OUTPUT

```
(14, 3)
[[102  89 117]
 [110  94 121]
 [115  93 118]]
```

`raw_dispatch_units` is the flat list of 42 readings (it is in the starter file). Part 6 explains `reshape`; for now, treat `dispatch` as built.

`dispatch.shape` is `(14, 3)` and it means something only because we decided **axis 0 is days** and **axis 1 is hubs** (East, Central, West). The array does not know this; we state it.

Row, column, cell

Python counts from 0, so **Day 2 is row index 1** and **West is column index 2**. Inside the brackets, `[row, column]`; a colon means *every position on that axis*.

p4_index.py

```
print(dispatch[1, :])   # all hubs, Day 2
print(dispatch[:, 2])   # West, every day
print(dispatch[1, 2])   # Day 2, West
```

OUTPUT

```
[110  94 121]
[117 121 118 119 125 109 104 116 119 120 126 131 112 106]
121
```

p4_week.py

```
week1 = dispatch[:7, :]
print(week1.shape)
```

OUTPUT

```
(7, 3)
```

`:7` takes rows 0 to 6 (seven days) and the second `:` keeps all three hubs, so the block is `(7, 3)`. A single integer removes that axis: `dispatch[:, 2]` is `(14,)`.

Boolean filtering

p4_mask_small.py

```
import numpy as np

grid = np.array([[2, 5, 8], [3, 6, 9]])
mask = grid > 5
print(mask)
print(grid[mask])
```

OUTPUT

```
[[False False  True]
 [False  True  True]]
[8 6 9]
```

A comparison is element-wise and keeps the grid's shape. Using the mask as an index gathers only the True values into a 1-D result.

p4_mask_dispatch.py

```
large_reading_mask = dispatch > 120
print(large_reading_mask.shape)
large_readings = dispatch[large_reading_mask]
print(large_readings, large_readings.shape)
```

OUTPUT

```
(14, 3)
[121 125 126 121 131] (5,)
```

Five readings are above 120: Day 2 West, Day 5 West, Day 11 West, Day 12 East, Day 12 West. The selected values no longer say which day or hub they came from; keep the mask if positions matter.

> **Watch out:** A fixed value such as 120 is not each hub's own benchmark. East's benchmark is 110, Central's 95. Comparing each hub with its own benchmark is Part 5.

Views and copies

p4_views.py

```
import numpy as np

sample = np.array([2, 5, 8])
slice_view = sample[:2]
filtered_copy = sample[sample > 4]

slice_view[0] = 20
filtered_copy[0] = 50
print(sample)
print(filtered_copy)
```

OUTPUT

```
[20  5  8]
[50  8]
```

A basic slice is a **view**: changing it changed `sample`. A Boolean selection is a **copy**. If you need an independent slice, use `.copy()`, e.g. `week1 = dispatch[:7, :].copy()`. (Lists behave differently: slicing a list always makes a copy.)

> TRY IT YOURSELF
> Using `dispatch`: (a) print all three hubs on Day 9; (b) print Central for every day; (c) print West for Days 8–14 and its shape; (d) use a mask to find the readings **below** 90 and how many there are.

Part 5 Vectorisation and broadcasting

Vectorisation means writing arithmetic once and letting it apply across the array, instead of looping value by value.

p5_vector.py

```
import numpy as np

example = np.array([[10, 20, 30],
                    [11, 21, 31]])
print(example + 2)
print(example + np.ones((2, 3)))
```

OUTPUT

```
[[12 22 32]
 [13 23 33]]
[[11. 21. 31.]
 [12. 22. 32.]]
```

A single number (a **scalar**) is used with every entry. Two arrays of the same shape combine matching entries. Note: `*` on arrays is element-wise, not matrix multiplication.

The business need: actual units against a benchmark

The **utilisation ratio** for one hub-day is units dispatched ÷ that hub's planned units. 1.00 means exactly on benchmark, 0.90 means 90% of it, 1.05 means 5% above. The same three benchmarks apply on every one of the

14 days, so copying them into 14 rows would be wasteful. NumPy can reuse the smaller array across the larger one: **broadcasting**.

The broadcasting rule

LINE THE SHAPES UP FROM THE RIGHT

```
dispatch          (14, 3)
daily_capacity        (3,)
---------------------------
result            (14, 3)
```

- Compare shapes starting from the **last** axis: 3 and 3.
- Two lengths are compatible if they are **equal**, or one of them is **1**.
- A missing leading axis behaves like length 1, so the `(3,)` vector is reused for each of the 14 rows.

p5_utilisation.py

```
utilisation = dispatch / daily_capacity
print(utilisation.shape)
print(utilisation[1, :])
print(np.round(utilisation[:3], 3))
```

OUTPUT

```
(14, 3)
[1.         0.98947368 1.00833333]
[[0.927 0.937 0.975]
 [1.    0.989 1.008]
 [1.045 0.979 0.983]]
```

Column 0 is always divided by 110, column 1 by 95, column 2 by 120. Day 2: East exactly on benchmark (1.0), Central slightly below, West slightly above.

When shapes do not align

p5_shape_error.py

```
one_per_day = np.arange(14)
print(dispatch / one_per_day)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s11_numpy\p5_shape_error.py", line 8, in <module>
    print(dispatch / one_per_day)
          ~~~~~~~~~^~~~~~~~~~~~~
ValueError: operands could not be broadcast together with shapes (14,3) (14,)
```

The line numbers in this error count a hidden setup block; the message is what matters.

Lined up from the right, 3 and 14 are not compatible. NumPy does not guess that 14 numbers mean days. The error is useful: it stops a calculation whose shapes do not describe the same thing.

Right shape, wrong meaning

p5_wrong_order.py

```
wrong_capacity = np.array([120, 95, 110])   # West, Central, East: wrong order!
print(dispatch[1, :] / wrong_capacity)
print(dispatch[1, :] / daily_capacity)
```

OUTPUT

```
[0.91666667 0.98947368 1.1       ]
[1.         0.98947368 1.00833333]
```

Watch out: The misordered version runs without any error, because the shapes are compatible. West now looks 10% above benchmark when it is really about 0.8% above. **Shape checks cannot catch errors in meaning.** Always keep the benchmark order the same as the column order.

Conditions broadcast too

p5_conditions.py

```
exceeded = dispatch > daily_capacity
at_capacity = dispatch == daily_capacity
at_or_above = dispatch >= daily_capacity

print(exceeded[1, :])
print("Strictly above:", exceeded.sum())
print("Exactly at:", at_capacity.sum())
print("At or above:", at_or_above.sum())
```

OUTPUT

```
[False False  True]
Strictly above: 14
Exactly at: 3
At or above: 17
```

On Day 2, East equals its benchmark (110 vs 110), so under `>` it is **False**. Three cells sit exactly on their benchmark. `>=` asks a different question: calling all 17 cells "above benchmark" would mislabel equality as an exceedance.

TRY IT YOURSELF

Each hub earns a different margin per unit dispatched: `margin = np.array([12.5, 14.0, 11.0])` (East, Central, West).

(a) Compute the margin earned per hub per day with one expression and print its shape. (b) Find how many hub-days were **below 95%** of their own benchmark. (c) Why would `np.array([12.5, 14.0])` fail?

Part 6 Reshape, split, stack

The supplied data is a flat list of 42 numbers in day order: East, Central, West for Day 1, then for Day 2, and so on. The values are all there; the structure is not.

p6_preview.py

```
import numpy as np

preview = np.array([102, 89, 117, 110, 94, 121])
print(preview.reshape(2, 3))
```

OUTPUT

```
[[102  89 117]
 [110  94 121]]
```

Six values become 2 rows of 3. The same idea gives the full grid: `dispatch = np.array(raw_dispatch_units).reshape(14, 3)`. The count must match: 14 × 3 = 42.

p6_reshape_error.py

```
print(np.array(raw_dispatch_units).reshape(13, 3))
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s11_numpy\p6_reshape_error.py", line 7, in <module>
    print(np.array(raw_dispatch_units).reshape(13, 3))
          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
ValueError: cannot reshape array of size 42 into shape (13,3)
```

Split into two weeks

p6_split.py

```
weeks = np.split(dispatch, 2, axis=0)
print([week.shape for week in weeks])
print("Week 1 total:", weeks[0].sum(), " Week 2 total:", weeks[1].sum())
```

OUTPUT

```
[(7, 3), (7, 3)]
Week 1 total: 2183  Week 2 total: 2231
```

Days are on axis 0, so `axis=0` splits the rows into two equal blocks, each keeping all three hubs. `axis=1` would try to split the three hub columns into two equal groups, which is impossible.

Stacking is a reconstruction check

p6_stack.py

```
rejoined = np.vstack((weeks[0], weeks[1]))
swapped = np.vstack((weeks[1], weeks[0]))

print(rejoined.shape, np.array_equal(rejoined, dispatch))
print(swapped.shape, np.array_equal(swapped, dispatch))
print("First row of swapped:", swapped[0])
```

OUTPUT

```
(14, 3) True
(14, 3) False
First row of swapped: [105  90 116]
```

Both have shape `(14, 3)`, but only one keeps the timeline in order: the swapped version starts with Day 8. **The right shape does not prove the right order**; check the values with `np.array_equal`.

TRY IT YOURSELF

Twelve monthly sales figures: `monthly = np.array([42, 38, 51, 47, 50, 55, 49, 53, 58, 61, 57, 66])` (in R thousands).

(a) Reshape them into 4 quarters × 3 months and print the shape. (b) Total each quarter. (c) Split the year into two halves along the quarter axis and print each half's total.

Part 7 Aggregation

An aggregation reduces many values to fewer: a total, a mean, a maximum. **The axis you name is the axis that disappears.**

p7_axis.py

```
first3 = dispatch[:3, :]
print(first3)
print("axis=0 (one total per hub):", first3.sum(axis=0))
print("axis=1 (one total per day):", first3.sum(axis=1))
```

OUTPUT

```
[[102  89 117]
 [110  94 121]
 [115  93 118]]
axis=0 (one total per hub): [327 276 356]
axis=1 (one total per day): [308 325 326]
```

Hub totals and daily totals

p7_totals.py

```
hub_totals = dispatch.sum(axis=0)
day_totals = dispatch.sum(axis=1)

print(hub_names)
print(hub_totals)
print("Network total:", dispatch.sum())
print(day_totals)
print("Mean:", round(day_totals.mean(), 2), " Min:", day_totals.min(), " Max:",
day_totals.max())
```

OUTPUT

```
['East Hub', 'Central Hub', 'West Hub']
[1481 1290 1643]
Network total: 4414
[308 325 326 326 342 285 271 311 328 323 341 355 293 280]
Mean: 315.29  Min: 271  Max: 355
```

`hub_totals[0]` belongs to East only because the column order is East, Central, West; the labels live in `hub_names`, not in the numbers.

From a value to its label: argmax

p7_argmax.py

```
busiest = day_totals.argmax()
print(busiest)
print(days[busiest])
print(day_totals[busiest])
print(dispatch.argmax())
```

OUTPUT

```
11
12
355
35
```

`max()` says how big the busiest day was; `argmax()` says **where** it is: position 11, which is **Day 12** because positions start at 0. Without an axis, `dispatch.argmax()` counts through all 42 cells, so 35 is the position of the single largest cell, not a day number.

Counting with Boolean arrays

p7_count.py

```
exceeded = dispatch > daily_capacity
print(exceeded.sum(axis=0))
print(exceeded.sum())
print(np.any(exceeded, axis=1).sum())
```

OUTPUT

```
[5 5 4]
14
7
```

True counts as 1, so a sum counts. 14 hub-day readings were above benchmark (East 5, Central 5, West 4), spread over 7 distinct days (`np.any(..., axis=1)` asks "did any hub exceed on this day?").

> **In data science:** West dispatched the most units (1,643) yet exceeded its own benchmark on the fewest days (4). **Total volume and benchmark pressure are different questions**, which is exactly why the axis and the comparison must match the question.

Spread, not just the average

p7_std.py

```
print(round(day_totals.std(), 2))
print(np.array([300, 300, 300]).std(), round(np.array([250, 300, 350]).std(), 2))
```

OUTPUT

```
24.19
0.0 40.82
```

The **standard deviation** describes how far days typically sit from the mean. Two sets with the same mean of 300 can have very different spreads.

Name	Meaning	Example	Shape
Scalar	one number, no axes	`dispatch.sum()` → 4414	`()`
Vector	values along one axis	`hub_totals` → [1481 1290 1643]	`(3,)`
Matrix	a rectangular grid	`dispatch`	`(14, 3)`

In data work a matrix usually has **rows as observations** (days) and **columns as features** (hubs). Machine-learning libraries expect exactly this layout.

> TRY IT YOURSELF
>
> (a) Total each hub for **Week 2** only. (b) Which hub had the lowest 14-day total? Use `argmin` and `hub_names`. (c) Which day had the lowest network total? Convert the position to a human day.

Part 8 Transfer: the same ideas on a photograph

A digital photo is also an array of numbers. This one is 720 pixels wide and 900 high; each pixel stores three numbers for red, green and blue. This is a short demonstration, separate from the practical.

p8_load.py

```
import numpy as np
from PIL import Image

image = np.array(Image.open("beach.jpg"))
print(image.shape, image.dtype)
print(image[300, 360, :])
```

OUTPUT

```
(900, 720, 3) uint8
[217 194 180]
```

Shape `(900, 720, 3)`: axis 0 is rows (height), axis 1 columns (width), axis 2 the three colour channels. Pillow reports size as (width, height) = (720, 900), but NumPy's shape starts with rows. `uint8` stores whole numbers from 0 to 255. Pillow is the image library Matplotlib already installs.

p8_crop.py

```
crop = image[250:550, 120:600, :]
print(crop.shape)

brightened = np.clip(image.astype(np.float32) * 1.20, 0, 255).astype(np.uint8)
print(image[300, 360, :], brightened[300, 360, :])
print("Values clipped at 255:", int((image.astype(np.float32) * 1.20 > 255).sum()))
```

OUTPUT

```
(300, 480, 3)
[217 194 180] [255 232 216]
Values clipped at 255: 61266
```

The crop uses the same slice rule as `dispatch[:7, :]`. To brighten safely: convert to float, multiply, **clip** to 0–255, then convert back. Red 217 × 1.2 = 260.4, which is clipped to 255. Kept in `uint8`, arithmetic would wrap around instead: 217 + 60 becomes 21.

p8_show.py

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(10, 4.4))
for ax, img, title in zip(axes, [image, crop, brightened],
                          ["Original (900, 720, 3)", "Crop (300, 480, 3)", "Brightened
x1.20"]):
    ax.imshow(img)
    ax.set_title(title, fontsize=10)
    ax.axis("off")
fig.tight_layout()
fig.savefig("photo_panels.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



The three arrays drawn with `imshow`: one expression changed every one of the 1,944,000 values.

Idea	Dispatch data	Photograph
Shape	`(14, 3)`: days × hubs	`(900, 720, 3)`: rows × columns × channels
Dtype	integer unit counts	`uint8`, 0 to 255
Slicing	`dispatch[:7, :]` → Week 1	`image[250:550, 120:600, :]` → a region
Vectorisation	`dispatch / daily_capacity`	`image.astype(np.float32) * 1.20`

Part 9 Charts with Matplotlib

Charts turn arrays into evidence. Choose the **question** first, then the arrays that answer it, then the chart. Three words sound alike:

Word	Meaning	In code
Figure	the whole canvas	`fig`
Axes	one chart area on the canvas, with its own x and y axes	`ax`
NumPy axis	a dimension of an array; nothing to do with chart axes	`sum(axis=0)`

Line chart: change over time

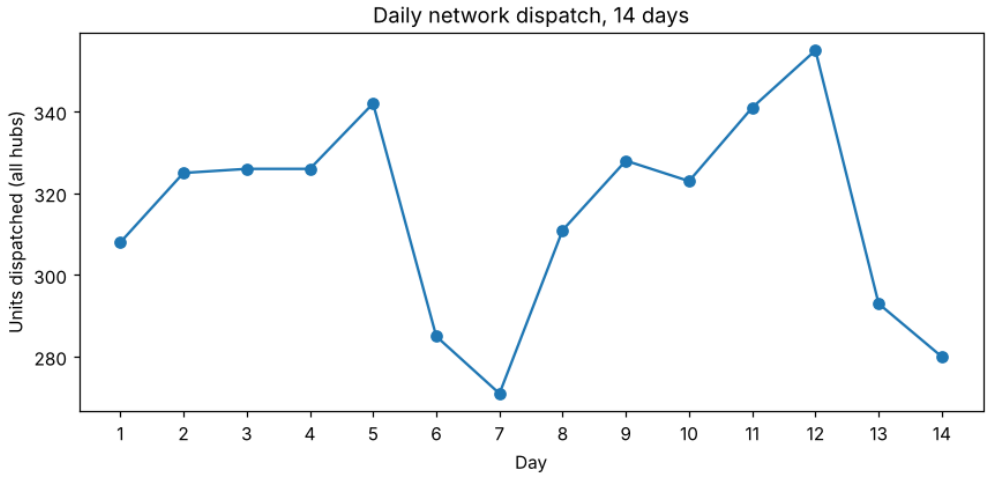
p9_line.py

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(days, day_totals, marker="o")
ax.set_xticks(days)
ax.set_title("Daily network dispatch, 14 days")
ax.set_xlabel("Day")
ax.set_ylabel("Units dispatched (all hubs)")
fig.tight_layout()
fig.savefig("line_daily.png", dpi=110)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Day 12 is the peak at 355 units; Day 7 the low at 271.

> **Watch out:** `ax.plot(day_totals)` without `days` labels the x axis 0 to 13, and the peak appears at 11 instead of 12. Always pass the human labels as x.

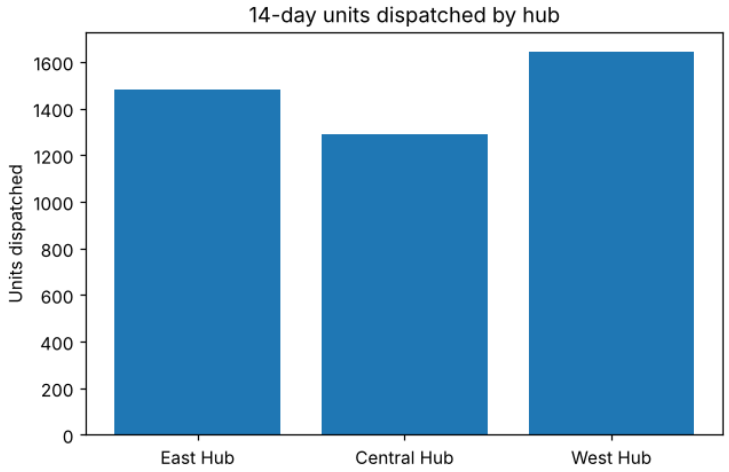
Bar chart: compare categories

p9_bar.py

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(hub_names, hub_totals)
ax.set_title("14-day units dispatched by hub")
ax.set_ylabel("Units dispatched")
fig.tight_layout()
fig.savefig("bar_hubs.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



West dispatched the most (1,643), then East (1,481), then Central (1,290).

Do not plot `daily_capacity` here: benchmarks are not totals and answer a different question.

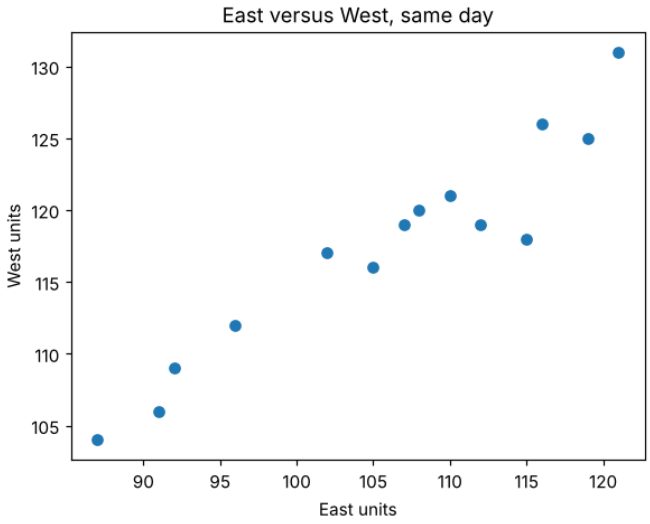
Scatter plot: a relationship between two measures

p9_scatter.py

```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
ax.scatter(dispatch[:, 0], dispatch[:, 2])
ax.set_title("East versus West, same day")
ax.set_xlabel("East units")
ax.set_ylabel("West units")
fig.tight_layout()
fig.savefig("scatter_east_west.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



Days with higher East dispatch usually have higher West dispatch: an association, not a cause.

Chart	Use it when	Needs
Line	order matters, e.g. days in sequence	paired x and y of equal length
Bar	comparing a few named categories	one total per category
Scatter	looking for a relationship between two measures	matched pairs, one per observation

Every chart needs a title that says what is plotted, axis labels with units, and x values that match what a reader expects.

TRY IT YOURSELF

Draw a line chart of **Central** dispatch over the 14 days, with a dashed horizontal line at its benchmark: `ax.axhline(95, linestyle="--")`. Give it a title and axis labels.

Part 10 Practical: the 14-day dispatch review

Trainer note: Learners complete `s11_dispatch_starter.py`, replacing each `...` with one expression. Insist on vectorised NumPy, not loops over days. When stuck, they print the shape of what they have.

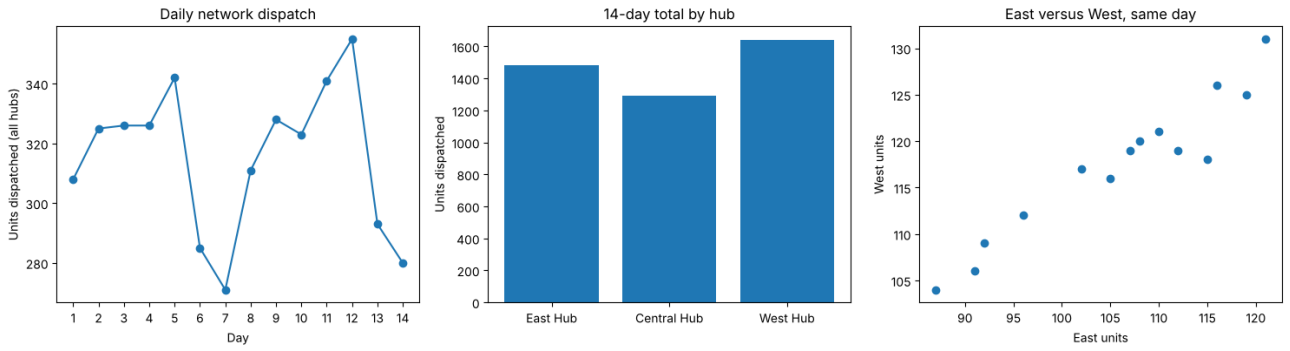
A retailer's operations manager wants to understand fourteen days of dispatch across three hubs. Answer four questions: How do 42 flat readings become a day-by-hub grid? On which days did a hub go above its own benchmark? What did each hub and the network dispatch, and which day was busiest? How do the two weeks compare, and what do three charts show?

TODO	You do	Check
1	Convert the flat list to an array with 14 day rows and 3 hub columns	shape `(14, 3)`, ndim 2, size 42; row 0 is Day 1
2	Select Day 2, the West hub, Days 1–7; a fixed-value mask `> 120`	`(3,)`, `(14,)`, `(7, 3)`; mask `(14, 3)`, 5 values
3	Broadcast the benchmark: ratio, strict `>` mask, equality mask	all `(14, 3)`; 14 above, 3 equal
4	Hub totals, day totals, mean/min/max, busiest day, counts above benchmark	busiest is Day 12 (355)
5	Split into weeks, total each, stack back, compare with `array_equal`	2183 and 2231; True
6a–6c	Choose the arrays for the line, bar and scatter charts	lengths 14/14, 3/3, 14/14

Reference run

TERMINAL

```
> python s11_dispatch_review.py
TODO 1: (14, 3) 2 42 row 0: [102  89 117]
TODO 2: [110  94 121] (14,) (7, 3) [121 125 126 121 131]
TODO 3: (14, 3) above: 14 equal: 3
TODO 4: hub totals [1481 1290 1643] network 4414
        mean 315.29 min 271 max 355
        busiest day 12 with 355 units
        days above benchmark per hub [5 5 4] | distinct days 7
TODO 5: week totals [2183, 2231] | rebuilt correctly: True
```



The three charts from TODO 6a–6c.

Interpret your evidence

Finding	Supported by
Network dispatch peaked on **Day 12 at 355 units** and was lowest on **Day 7 at 271**. The average day was about 315.29 units (std ≈ 24.19).	day totals, argmax, line chart
West dispatched the most (1,643) but exceeded its benchmark on the fewest days (4). East and Central each exceeded theirs on 5. In total 14 hub-days over 7 distinct days.	hub totals, exceedance counts, bar chart
Week 2 (2,231) dispatched 48 units more than Week 1 (2,183).	split and totals

> **Watch out: One limitation, stated plainly:** these are 42 synthetic counts compared with nominal planning figures. They cannot show *why* a day was busy or whether a hub failed. Being above a nominal benchmark is not a service failure, and the scatter shows association, not causation. Fourteen days is a short window.

Close

Explain it in your own words

1. In a `(14, 3)` array, which axis does `sum(axis=0)` remove, and what is left?
2. Why can three benchmark values be compared with all 14 days at once?
3. What do the dispatch matrix and the photograph have in common?
4. Which chart answers "how did dispatch change over time?", and what must it not claim?
5. Give one situation where a list is the better choice than an array, and one where the array is better.

What comes next

Our array needed us to remember that column 0 is East. Real tables carry **named columns**, **mixed types** and **missing values**. Session 12 introduces pandas, which is built on NumPy and answers exactly these needs. NumPy returns later for statistics, vectors and matrices, and image learning.

Appendix: Answers to the exercises

Part 0

a0.py

```
stock = {"bread": 40, "milk": 25, "eggs": 60}
stock["butter"] = 15
stock["milk"] = 30
removed = stock.pop("eggs")
print("Removed eggs:", removed)
reorder = {item: qty < 30 for item, qty in stock.items()}
print(reorder)

branch_a = {"C01", "C02", "C03", "C05"}
branch_b = {"C02", "C04", "C05"}
print(sorted(branch_a & branch_b), sorted(branch_a | branch_b), sorted(branch_a -
branch_b))
```

OUTPUT

```
Removed eggs: 60
{'bread': False, 'milk': False, 'butter': True}
['C02', 'C05'] ['C01', 'C02', 'C03', 'C04', 'C05'] ['C01', 'C03']
```

Part 2

a2.py

```
import numpy as np

sales = [42000, 38500, 51000, 47250]
costs = [30000, 29000, 33500, 31000]

profit = [s - c for s, c in zip(sales, costs)]
print(profit, [p for p in profit if p > 15000])

sales_a, costs_a = np.array(sales), np.array(costs)
profit_a = sales_a - costs_a
print(profit_a, profit_a[profit_a > 15000])
```

OUTPUT

```
[12000, 9500, 17500, 16250] [17500, 16250]
[12000  9500 17500 16250] [17500 16250]
```

(c) A list only supports `*` with a whole number, which repeats it; an array applies the multiplication to every element.

Part 3

a3.py

```
import numpy as np

for arr in [np.zeros((3, 4)), np.arange(1, 32), np.linspace(0, 100, 5), np.array([[1, 2],
[3, 4], [5, 6]])]:
    print(arr.shape, arr.ndim, arr.size)
```

OUTPUT

```
(3, 4) 2 12
(31,) 1 31
(5,) 1 5
(3, 2) 2 6
```

Part 4

a4.py

```
print(dispatch[8, :])
print(dispatch[:, 1])
print(dispatch[7:, 2], dispatch[7:, 2].shape)
low = dispatch[dispatch < 90]
print(low, low.size)
```

OUTPUT

```
[112  97 119]
[ 89  94  93 100  98  84  80  90  97  95  99 103  85  83]
[116 119 120 126 131 112 106] (7,)
[89 84 87 80 85 83] 6
```

Part 5

a5.py

```
margin = np.array([12.5, 14.0, 11.0])
earned = dispatch * margin
print(earned.shape, earned[0])
print((dispatch < 0.95 * daily_capacity).sum())
```

OUTPUT

```
(14, 3) [1275. 1246. 1287.]
15
```

(c) A `(2,)` vector cannot line up with the 3 hub columns: 2 and 3 are not equal and neither is 1.

Part 6

a6.py

```
import numpy as np

monthly = np.array([42, 38, 51, 47, 50, 55, 49, 53, 58, 61, 57, 66])
quarters = monthly.reshape(4, 3)
print(quarters.shape)
print(quarters.sum(axis=1))
halves = np.split(quarters, 2, axis=0)
print([int(half.sum()) for half in halves])
```

OUTPUT

```
(4, 3)
[131 152 160 184]
[283, 344]
```

Part 7

a7.py

```
week2 = dispatch[7:, :]
print(week2.sum(axis=0))
hub_totals = dispatch.sum(axis=0)
print(hub_names[hub_totals.argmin()])
day_totals = dispatch.sum(axis=1)
print("Quietest day:", days[day_totals.argmin()], day_totals.min())
```

OUTPUT

```
[749 652 830]
Central Hub
Quietest day: 7 271
```

Part 9

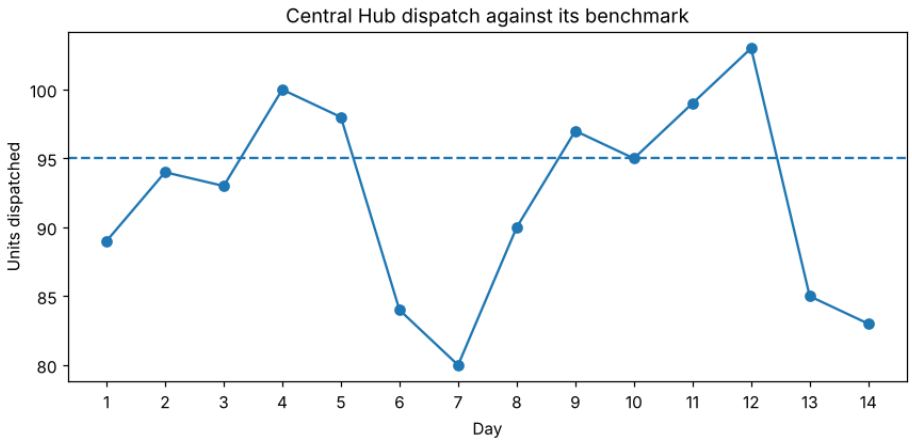
a9.py

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(days, dispatch[:, 1], marker="o")
ax.axhline(95, linestyle="--")
ax.set_xticks(days)
ax.set_title("Central Hub dispatch against its benchmark")
ax.set_xlabel("Day")
ax.set_ylabel("Units dispatched")
fig.tight_layout()
fig.savefig("central_line.png", dpi=110)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Part 10: completed s11_dispatch_review.py

s11_dispatch_solution.py

```python
# Session 11 -- 14-day dispatch review (completed)
import numpy as np
import matplotlib.pyplot as plt

raw_dispatch_units = [
    102, 89, 117,
    110, 94, 121,
    115, 93, 118,
    107, 100, 119,
    119, 98, 125,
    92, 84, 109,
    87, 80, 104,
    105, 90, 116,
    112, 97, 119,
    108, 95, 120,
    116, 99, 126,
    121, 103, 131,
    96, 85, 112,
    91, 83, 106,
]
hub_names = ["East Hub", "Central Hub", "West Hub"]
days = np.arange(1, 15)
daily_capacity = np.array([110, 95, 120])   # East, Central, West

# TODO 1: make the matrix
dispatch = np.array(raw_dispatch_units).reshape(14, 3)
print("TODO 1:", dispatch.shape, dispatch.ndim, dispatch.size, "row 0:", dispatch[0])

# TODO 2: select and filter (fixed value 120)
day2 = dispatch[1, :]
west = dispatch[:, 2]
week1 = dispatch[:7, :]
large_mask = dispatch > 120
print("TODO 2:", day2, west.shape, week1.shape, dispatch[large_mask])

# TODO 3: broadcast each hub's own benchmark
utilisation = dispatch / daily_capacity
exceeded = dispatch > daily_capacity
at_capacity = dispatch == daily_capacity
print("TODO 3:", utilisation.shape, "above:", exceeded.sum(), "equal:", at_capacity.sum())
```

```
# TODO 4: reduce an axis
hub_totals = dispatch.sum(axis=0)
day_totals = dispatch.sum(axis=1)
busiest = day_totals.argmax()
print("TODO 4: hub totals", hub_totals, "network", dispatch.sum())
print("        mean", round(day_totals.mean(), 2), "min", day_totals.min(), "max",
day_totals.max())
print("        busiest day", days[busiest], "with", day_totals[busiest], "units")
print("        days above benchmark per hub", exceeded.sum(axis=0),
      "| distinct days", np.any(exceeded, axis=1).sum())

# TODO 5: weeks
weeks = np.split(dispatch, 2, axis=0)
week_totals = [int(week.sum()) for week in weeks]
rejoined = np.vstack((weeks[0], weeks[1]))
print("TODO 5: week totals", week_totals, "| rebuilt correctly:", np.array_equal(rejoined,
dispatch))

# TODO 6a-6c: charts
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
axes[0].plot(days, day_totals, marker="o")                # 6a line: 14 and 14
axes[0].set_xticks(days)
axes[0].set_title("Daily network dispatch")
axes[0].set_xlabel("Day")
axes[0].set_ylabel("Units dispatched (all hubs)")
axes[1].bar(hub_names, hub_totals)                        # 6b bar: 3 and 3
axes[1].set_title("14-day total by hub")
axes[1].set_ylabel("Units dispatched")
axes[2].scatter(dispatch[:, 0], dispatch[:, 2])           # 6c scatter: 14 and 14
axes[2].set_title("East versus West, same day")
axes[2].set_xlabel("East units")
axes[2].set_ylabel("West units")
fig.tight_layout()
fig.savefig("dispatch_review.png", dpi=100)
plt.show()
```